

IVOR Paine Memorial Hospital

Database Normalization Report

CL2005 - Database Systems Lab Project | Phase 2

Group Members

Muhammad Mustafa (24i-0525)

Anas Mumtaz (24i-0653)

Abdul Moeed (24i-0720)

Muhammad Mughees Tariq Khawaja (24i-0806)

1. Overview

We designed the schema for the IVOR Paine Memorial Hospital database using Entity-Relationship modelling principles, working from the EERD outward to the relational mapping. Throughout this process we identified and eliminated redundancies at each stage, which resulted in all tables satisfying **Third Normal Form (3NF) / BCNF** by the time the final schema was produced. This report explains the problems we found at each stage and the decompositions we performed to resolve them.

2. Functional Dependencies

Before beginning normalization, we identified the following functional dependencies across the core entities of the schema:

Determinant	Dependent Attributes
PatientNo	PatientName, DateOfBirth, Address, CareUnitNo, BedNo, ConsultantNo, DoctorInChargeNo
CareUnitNo	CareUnitName, Capacity, WardName
WardName	Location, SpecialtyName
SpecialtyName	Description

BedNo	BedType, IsOccupied, WardName
ComplaintCode	Description, Severity
TreatmentCode	Description, TreatmentType, Duration
DoctorNo	DoctorName, Address, DateOfBirth
(PatientNo, ComplaintCode, TreatmentCode, DateStarted)	DoctorNo, DateEnded, Notes
(DoctorNo, DateJoined)	EndDate, PositionCode, ConsultantNo
(DoctorNo, FromDate, PositionCode)	ToDate, Establishment
(DoctorNo, GradeDate)	PerformanceGrade
NurseID	NurseName, DateOfBirth, Address, PositionCode

3. Step 0 - Unnormalized Form (UNF)

As a starting point, we considered what the data would have looked like had it all been stored in a single flat table, which is typically how raw data is first collected before any normalization takes place:

PATIENT_TREATMENT_UNF

Attribute	Notes
PatientNo, PatientName, DOB, Address	Patient details
PhoneNumbers*	Multi-valued - not atomic
CareUnitNo, WardName, BedNo	Location info
ConsultantNo, DoctorInChargeNo	Assigned doctors
ComplaintCode, ComplaintDesc, Severity	Repeating group - one row per complaint
TreatmentCode, TreatmentDesc, TreatmentType	Repeating group - one row per treatment
DoctorNo, DoctorName	Treating doctor (repeats)
DateStarted, DateEnded	Treatment dates (repeating)

Problems we identified

- We noticed that the treatment-related columns (ComplaintCode, ComplaintDesc, TreatmentCode, TreatmentDesc, DoctorNo, DoctorName, DateStarted, DateEnded) formed a repeating group, since a single patient can have multiple complaints and multiple treatments. This meant multiple rows per patient with duplicated personal details.
- The PhoneNumbers attribute was non-atomic, as a single patient could have more than one phone number stored in the same cell, directly violating the atomicity requirement.
- We also observed that any change to a doctor's name would require updating every treatment row that referenced that doctor - a clear update anomaly.
- Additionally, it was impossible to record a patient complaint without simultaneously recording a treatment, creating an insertion anomaly.

4. Step 1 - First Normal Form (1NF)

To bring the table into 1NF, we needed to ensure all attribute values were atomic and that no repeating groups existed. We performed the following decompositions:

What we did

- We separated PhoneNumbers into its own table, PATIENT_PHONE_1NF, with a composite primary key of (PatientNo, PhoneNumber). This eliminated the non-atomic column and allowed each phone number to occupy its own row.
- We extracted the repeating treatment group into TREATMENT_RECORD_1NF, giving it a composite primary key of (PatientNo, ComplaintCode, TreatmentCode, DateStarted) to uniquely identify each treatment instance.
- The remaining patient-level attributes stayed in PATIENT_1NF with PatientNo as the primary key.

This resulted in three 1NF-compliant tables with no repeating groups and fully atomic values:

Table	Columns	Primary Key
PATIENT_1NF	PatientNo, PatientName, DOB, Address, CareUnitNo, WardName, BedNo, ConsultantNo, DoctorInChargeNo	PatientNo
PATIENT_PHONE_1NF	PatientNo, PhoneNumber	(PatientNo, PhoneNumber)
TREATMENT_RECORD_1NF	PatientNo, ComplaintCode, ComplaintDesc, Severity, TreatmentCode, TreatmentDesc, TreatmentType, DoctorNo, DoctorName, DateStarted, DateEnded	(PatientNo, ComplaintCode, TreatmentCode, DateStarted)

5. Step 2 - Second Normal Form (2NF)

With the tables in 1NF, we then examined TREATMENT_RECORD_1NF for partial dependencies - cases where a non-key attribute depended on only part of the composite primary key rather than the whole key. The primary key was (PatientNo, ComplaintCode, TreatmentCode, DateStarted).

Partial dependencies we found and removed

Primary Key: (PatientNo, ComplaintCode, TreatmentCode, DateStarted)

Partial Key Dependency	Violating Attributes	Action
ComplaintCode	ComplaintDesc, Severity	Extract to COMPLAINT table
TreatmentCode	TreatmentDesc, TreatmentType	Extract to TREATMENT table
DoctorNo	DoctorName	Extract to DOCTOR table

We split TREATMENT_RECORD_1NF into four tables. Each partial dependency was moved into its own dedicated table, and TREATMENT_RECORD_2NF retained only the foreign keys and the attributes that genuinely depended on the full composite key:

Tables after 2NF

Table	Columns	Primary Key
TREATMENT_RECORD_2NF	PatientNo, ComplaintCode, TreatmentCode, DoctorNo, DateStarted, DateEnded	(PatientNo, ComplaintCode, TreatmentCode, DateStarted)
COMPLAINT	ComplaintCode, Description, Severity	ComplaintCode
TREATMENT	TreatmentCode, Description, TreatmentType, Duration	TreatmentCode
DOCTOR_2NF	DoctorNo, DoctorName	DoctorNo

6. Step 3 - Third Normal Form (3NF)

After achieving 2NF, we examined the remaining tables for transitive dependencies - situations where a non-key attribute determined another non-key attribute. We found one clear violation in PATIENT_1NF:

Transitive dependency we found in PATIENT_1NF

CareUnitNo WardName - WardName was present as a non-key column in PATIENT_1NF, but it was not directly determined by PatientNo. Instead, it was determined by CareUnitNo, which is itself a non-key attribute relative to PatientNo.

This gave us a transitive chain: **PatientNo CareUnitNo WardName**. Having WardName in the Patient table meant that if a care unit was reassigned to a different ward, we would have had to update every patient row belonging to that unit - a textbook update anomaly caused by the transitive dependency.

What we did to resolve it

- We removed WardName from the Patient table entirely. The Patient table now holds only CareUnitNo as a foreign key, letting the CareUnit table own the ward relationship.
- We created a dedicated CareUnit table containing CareUnitNo, CareUnitName, Capacity, WardName, and NurseInChargeID. This broke the transitive chain - WardName now depends directly on CareUnitNo, which is the primary key of CareUnit.

- The Ward table was kept separate with WardName as its own primary key, holding Location and SpecialtyName, so that ward-level details are stored exactly once.

This decomposition eliminated the transitive dependency and resulted in the following 3NF-compliant tables:

Table	Columns	Primary Key
PATIENT (final)	PatientNo, PatientName, DOB, Address, CareUnitNo, BedNo, ConsultantNo, DoctorInChargeNo	PatientNo
CARE_UNIT	CareUnitNo, CareUnitName, Capacity, WardName, NurseInChargeID	(CareUnitNo, WardName)
WARD	WardName, Location, SpecialtyName	WardName
SPECIALTY	SpecialtyName, Description	SpecialtyName

7. BCNF Verification

After completing the 3NF decompositions, we verified that every table in the schema also satisfies BCNF - meaning for every functional dependency $X \rightarrow Y$, X must be a superkey. We went through each significant table and confirmed no violations remained:

Table	Functional Dependency	BCNF Status
Patient	PatientNo $\rightarrow$ all attributes	Satisfies BCNF
TreatmentRecord	(PatientNo, ComplaintCode, TreatmentCode, DateStarted) $\rightarrow$ DoctorNo, DateEnded, Notes	Satisfies BCNF
DoctorTeamRecord	(DoctorNo, DateJoined) $\rightarrow$ ConsultantNo, PositionCode, EndDate	Satisfies BCNF
CareUnit	(CareUnitNo, WardName) $\rightarrow$ CareUnitName, Capacity, NurseInChargeID	Satisfies BCNF
PerformanceHistory	(DoctorNo, GradeDate) $\rightarrow$ PerformanceGrade	Satisfies BCNF
PreviousExperience	(DoctorNo, FromDate, PositionCode) $\rightarrow$ ToDate, Establishment	Satisfies BCNF
NightSister / DaySister	NurseID $\rightarrow$ WardName, ShiftStart, ...	Satisfies BCNF

8. Conclusion

Through the normalization process documented above, we brought the hospital database schema from an unnormalized flat structure to a fully **3NF and BCNF compliant** design across all 25 tables. At the 1NF stage, we separated multi-valued attributes and eliminated repeating groups. At 2NF, we decomposed TREATMENT_RECORD_1NF by extracting Complaint, Treatment, and Doctor as independent tables, removing all partial dependencies on the composite key. At 3NF, we resolved the transitive dependency CareUnitNo WardName by isolating the CareUnit entity, ensuring no non-key attribute determined another. The final schema contains no redundancy, no update anomalies, and no insertion or deletion anomalies - each fact is stored exactly once, in exactly the right place.